PES-VCS Lab Report

Implementation Overview

PES-VCS is a local version control system built from scratch in C, modelled after Git's internal design. It implements content-addressable object storage, tree hierarchy construction, a text-based staging area, and commit history with parent-pointer linked lists.

Phase 1 — Object Storage

What Was Implemented

`object_write` builds a full object by prepending a header (`"<type> <size>\0"`) to the raw data, computes a SHA-256 hash of the whole thing, deduplicates by checking existence, shards into `.pes/objects/XX/`, writes atomically via a temp file + `fsync` + `rename`, and syncs the shard directory.

`object_read` reconstructs the file path from the hash, reads the file, recomputes SHA-256 to verify integrity (returns `-1` on mismatch), parses the header to extract type and size, and returns a heap-allocated copy of the data portion.

Screenshot 1A — `./test_objects` output

```
ramesh@DESKTOP-H0GQBG3:~/PES2UG24CS385-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B — Sharded object store

```
ramesh@DESKTOP-H0GQBG3:~/PES2UG24CS385-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2 — Tree Objects

What Was Implemented

`tree_from_index` loads the index and calls a recursive helper `write_tree_level(entries, count, prefix, id_out)`. The helper iterates index entries that share a given path prefix. If an entry has no `/` after the prefix it is a direct file and becomes a blob `TreeEntry`. If it has a `/`, the subdirectory name is extracted, a new prefix is formed, and the helper recurses. After processing all entries at a level, `tree_serialize` and `object_write(OBJ_TREE, ...)` persist the tree, and the hash is returned up the call stack.

Screenshot 2A – `./test_tree` output

```
ramesh@DESKTOP-H0GQBG3:~/PES2UG24CS385-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B – Raw binary tree object (od hex dump)

```
$ od -A x -t x1z
.pes/objects/34/fb83796d921969e6ba2a0c30dac825ea371b12576ff5709ffb9064f2112da4
| head -20
000000 74 72 65 65 20 39 38 00 31 30 30 36 34 34 20 66  >tree 98.100644 f<
000010 69 6c 65 31 2e 74 78 74 00 b4 e7 6a ee 38 85 48  >ile1.txt...j.8.H<
000020 39 55 7e 5a 31 32 e3 46 ad bf 82 88 ec 2b a8 78  >9U~Z12.F.....+.x<
000030 f2 cd 91 5d 5c 23 26 af 00 31 30 30 36 34 34 20  >...]\\#&..100644 <
000040 66 69 6c 65 32 2e 74 78 74 00 e0 0c 50 e1 6a 2d  >file2.txt...P.j-<
000050 f3 8f 8d 6b f8 09 e1 81 ad 02 48 da 6e 67 19 f3  >...k......H.ng..<
000060 5f 9f 7e 65 d6 f6 06 19 9f 7f                    >_.~e......<
```

You can see the ASCII header `tree 98\0`, then the first entry `100644 file1.txt\0` followed by 32 raw bytes of SHA-256 hash, then the second entry similarly.

Phase 3 – Index (Staging Area)

What Was Implemented

`index_load` opens `.pes/index` for reading (returns an empty index if the file doesn't exist). Each line is parsed with `sscanf` using the format `"%o %64s %llu %u %511s"` to extract mode, hex hash, mtime, size, and path. `hex_to_hash` converts the hex string to an `ObjectID`.

`index_save` heap-allocates a copy of the index (to avoid stack overflow with 10,000-entry structs), sorts by path with `qsort`, writes to a temp file via `fprintf`, calls `fflush`/`fsync`/`fclose`, then `rename`s atomically over the real index file.

`index_add` reads the file contents with `fopen`/`fread`, writes them as a blob via `object_write(OBJ_BLOB, ...)`, calls `lstat` for metadata, and either updates an existing index entry (found via `index_find`) or appends a new one, then calls `index_save`.

Screenshot 3A – `pes init → pes add → pes status`

```
rameshh@DESKTOP-H0GQBG3:~/PES2UG24CS385-pes-vcs$ rm -rf .pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  README.md
  untracked:  test_tree
  untracked:  pes.c
  untracked:  commit.h
  untracked:  test_objects.c
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  tree.c
  untracked:  Makefile
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  test_objects
  untracked:  object.c
  untracked:  index.c
  untracked:  pes.h

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776753809 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776753809 6 file2.txt
```

Screenshot 3B — `cat .pes/index`

```
cat .pes/index
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt
```

Phase 4 — Commits and History

What Was Implemented

`commit_create` calls `tree_from_index` to get the root tree hash, attempts `head_read` to get the parent commit (skipped for the first commit by checking the return value), fills a `Commit` struct with the tree hash, parent, author string from `pes_author()`, and `time(NULL)` timestamp, serializes it with `commit_serialize`, writes it with `object_write(OBJ_COMMIT, ...)`, and finally calls `head_update` to atomically advance the branch pointer.

Screenshot 4A — `pes log` with three commits

```
./pes log
find .pes -type f | sort
cat .pes/refs/heads/main
cat .pes/HEAD
Initialized empty PES repository in .pes/
Committed: 624e9a74c302... Initial commit
Committed: 2af12d832e86... Add world
Committed: 6eece429b3cd... Add farewell
commit 6eece429b3cd707699b9016751b3e8c58bfc534af0c56ed1b076cbfcc510bb71
Author: Your Name <PES2UG24CS385>
Date:   1776753842

    Add farewell

commit 2af12d832e864b4348cf6080b84144a25e888bc466532ee625849e580967f4cb
Author: Your Name <PES2UG24CS385>
Date:   1776753842

    Add world

commit 624e9a74c3027895971a7d6ef7148159752bbb60660a937692a5be9da6a901bd
Author: Your Name <PES2UG24CS385>
Date:   1776753842

    Initial commit
```

Screenshot 4B — Object store growth after three commits

```
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2a/f12d832e864b4348cf6080b84144a25e888bc466532ee625849e580967f4cb
.pes/objects/62/4e9a74c3027895971a7d6ef7148159752bbb60660a937692a5be9da6a901bd
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/ece429b3cd707699b9016751b3e8c58bfc534af0c56ed1b076cbfcc510bb71
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
6eece429b3cd707699b9016751b3e8c58bfc534af0c56ed1b076cbfcc510bb71
ref: refs/heads/main
ramesh@DESKTOP-H0GQBG3:~/PES2UG24CS385-pes-vcs$ make test-integration
```

Screenshot 4C — Reference chain

```
find .pes -type f | sort
cat .pes/refs/heads/main
cat .pes/HEAD
Initialized empty PES repository in .pes/
Committed: 624e9a74c302... Initial commit
Committed: 2af12d832e86... Add world
Committed: 6eece429b3cd... Add farewell
commit 6eece429b3cd707699b9016751b3e8c58bfc534af0c56ed1b076cbfcc510bb71
Author: Your Name <PES2UG24CS385>
Date:   1776753842
```

Full Integration Test

```
ramesh@DESKTOP-H0GQBG3:~/PES2UG24CS385-pes-vcs$ make test-integration
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 060f5b8741de... Initial commit

Log after first commit:
commit 060f5b8741de66f02a2cb51e9e0aa89d1850856262883593db9d161857672275
Author: Your Name <PES2UG24CS385>
Date:   1776753886

    Initial commit


--- Second Commit ---
Committed: 1ff81c063861... Update file.txt

--- Third Commit ---
```

```
--- Full History ---
commit ffa7b9a43e1efba3dfa2947cddd83bd981ddb9bf5783ffee62321483b4acc094
Author: Your Name <PES2UG24CS385>
Date:   1776753887

    Add farewell

commit 1ff81c0638610ca3d5321c03c781fada7a1183b52376fb16fa5051673cf08371
Author: Your Name <PES2UG24CS385>
Date:   1776753886

    Update file.txt

commit 060f5b8741de66f02a2cb51e9e0aa89d1850856262883593db9d161857672275
Author: Your Name <PES2UG24CS385>
Date:   1776753886

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
ffa7b9a43e1efba3dfa2947cddd83bd981ddb9bf5783ffee62321483b4acc094

--- Object Store ---
Objects created:
10
.pes/objects/06/0f5b8741de66f02a2cb51e9e0aa89d1850856262883593db9d161857672275
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1f/f81c0638610ca3d5321c03c781fada7a1183b52376fb16fa5051673cf08371
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/ff/a7b9a43e1efba3dfa2947cddd83bd981ddb9bf5783ffee62321483b4acc094

=== All integration tests completed ===
```

Phase 5 — Branching and Checkout (Analysis)

Q5.1 — How would you implement `pes checkout <branch>`?

`pes checkout <branch>` needs to do two things: update the repository metadata and update the working directory.

Repository metadata changes in `.pes/`:
- Rewrite `HEAD` to contain `ref: refs/heads/<branch>` (or create the branch file first if switching to a new branch). This is a single atomic write using the temp-file-then-rename pattern.

Working directory update:
1. Read the target branch's commit hash from `.pes/refs/heads/<branch>`.
2. Parse that commit object to get its root tree hash.
3. Walk the tree recursively: for every blob entry, read the blob from the object store and overwrite the corresponding working file with its contents. For subtrees, create directories as needed.
4. Files that exist in the current tree but not in the target tree must be deleted.
5. Update the index to match the target tree exactly (each entry's hash, mtime, size).

What makes it complex:

The hard part is handling files the user has modified but not staged. You need to detect these "dirty" files and refuse to overwrite them (otherwise the user loses work). This requires a three-way comparison per file: current working file vs index entry vs target tree entry. If the working file differs from the index (user has local edits) AND the target tree has a different version of the same file, the checkout must abort with an error like `"error: Your local changes to 'foo.c' would be overwritten by checkout"`. Handling new files that only exist in one branch, and correctly managing directory creation and deletion, adds further edge cases.

Q5.2 — Detecting dirty working directory conflicts

For each file that appears in **both** the current branch's tree and the target branch's tree but with **different blob hashes**:

1. Look up the file's entry in the index (using `index_find`).
2. Call `stat()` on the working file and compare `st_mtime` and `st_size` against the stored `mtime_sec` and `size` in the index entry.
3. If either differs, the file has been modified since it was last staged — it is "dirty."
4. A dirty file whose blob hash differs between the two branches is a conflict: checkout must refuse.

For absolute certainty (since mtime can be unreliable in some scenarios), you can re-hash the working file with `object_write` or `compute_hash` and compare directly against the index's stored `ObjectID`. This is slower but eliminates false negatives from timestamp collisions.

Files that only exist in one branch and are also present in the working directory (but untracked) should also be checked: if the target branch would create a file at a path that already exists as an untracked working file, checkout should refuse to overwrite it.

Q5.3 — Detached HEAD and Recovery

In detached HEAD state, `HEAD` contains a raw commit hash (`abc123...`) instead of a branch reference (`ref: refs/heads/main`). When you make new commits, `head_update` writes the new commit hash directly into `HEAD`. No branch file is ever updated, so the new commits have no named reference pointing to them.

If you then run `pes checkout main`, `HEAD` is rewritten to `ref: refs/heads/main`, and the detached commits are now **unreachable** — no branch, tag, or HEAD points to them. They will be collected by garbage collection eventually.

Recovery: If you still have the commit hash (from terminal scrollback, or from `pes log` output taken before switching), you can recover by creating a new branch pointing to it:

```bash
echo "<lost-commit-hash>" > .pes/refs/heads/recovery
```

Then checkout that branch. This is exactly what Git's `git branch recovery <hash>` does. Without the hash, recovery is only possible if GC hasn't run yet — you could scan all objects in `.pes/objects/` looking for commit objects, parse them, and reconstruct the dangling chain.

Phase 6 — Garbage Collection (Analysis)

Q6.1 — Algorithm to Find and Delete Unreachable Objects

Algorithm (mark-and-sweep):

1. **Mark phase** — collect all reachable object hashes into a hash set:
   - Start from every file in `.pes/refs/heads/` (each is a commit hash).
   - For each commit: mark it reachable, parse it to get its tree hash and parent hash.
   - Recursively walk the tree: mark the tree object, then for each entry, if it is a blob mark it, if it is a subtree recurse into it.
   - Follow the parent pointer and repeat until a commit with no parent is reached.
   - Repeat for all branches.

2. **Sweep phase** — delete unmarked objects:
   - `find .pes/objects -type f` to enumerate all stored objects.
   - Convert each filename (shard dir + filename) back to a 64-char hex hash.
   - If the hash is not in the reachable set, delete the file.
   - Remove any now-empty shard directories.

Data structure: A hash set of `ObjectID` values (32-byte binary hashes). In C, a sorted array of hashes with `bsearch` works well and requires only `32 × N` bytes. For large repos, a proper hash table gives O(1) lookup instead of O(log N).

Estimate for 100,000 commits, 50 branches:
Assuming an average of 10 unique files per commit (many files are shared across commits via deduplication), and one tree object per commit:
- Commits: ~100,000
- Tree objects: ~100,000 (root trees) + subtrees ≈ ~200,000

- Blob objects: 10 unique blobs/commit × 100,000 ≈ ~1,000,000 (with deduplication, much fewer in practice)

In the worst case (no sharing), you visit roughly **1.2 million objects**. With typical deduplication, the reachable set is much smaller — perhaps 50,000–200,000 unique blobs.

Q6.2 — GC Race Condition with Concurrent Commits

The race condition:

1. A `commit` operation calls `object_write` to store a new **blob** (the contents of a staged file). The blob is now on disk in `.pes/objects/`.
2. GC runs at this exact moment. It traverses all branches and their commit chains. The new blob has not yet been referenced by any tree or commit object, so GC marks it **unreachable** and deletes it.
3. The `commit` operation continues: it calls `tree_from_index`, which calls `object_write(OBJ_TREE, ...)` referencing the blob's hash. It then calls `object_write(OBJ_COMMIT, ...)`. The commit object is written and `head_update` is called — but the blob the tree points to no longer exists. The repository is now **corrupt**.

How Git avoids this:

Git's real GC (`git gc`) uses a **grace period**: any object whose file modification time is newer than a configurable threshold (default: 2 weeks for loose objects) is treated as reachable regardless of whether any reference points to it. An in-progress `git commit` takes milliseconds, not weeks, so its intermediate objects are always protected.

Additionally, Git writes a `.git/gc.pid` lockfile before starting GC, preventing two GC processes from running concurrently. The lock is checked and any existing lock from a live PID causes GC to abort. Git also writes `FETCH_HEAD`, `MERGE_HEAD`, and other temporary refs that keep intermediate objects reachable during multi-step operations.

In PES-VCS, the simplest safe approach would be: before GC deletes any object, check `st_mtime` — skip deletion if the object is younger than a grace period (e.g., 60 seconds). A more robust approach would use an advisory lock file (`.pes/GC_LOCK`) that `commit_create` holds for its duration, and GC refuses to run while the lock exists.